

EE 542 – Laboratory #2: Fast, Reliable File Transfer

Instructor: Young H. Cho

This lab should be done as a GROUP.

What you will submit is a YouTube video log of your team doing and presenting the work. At the end of every day you spend working, you will need to use your phones to record all members (in person) giving a short summary of what you did that day/time, the problems that EACH person in the team ran into, and your fixes. You will need to include a video of yourselves navigating the screen and showing progress. This must not be a screen-capturing tool, but rather the phone recording of the display. The videos must be stitched together and edited before submission.

You must create a GitHub repository for your group and use GitHub for all development. When a member pushes their contribution to the repository, he/she must include a clear description of the updates. At the end of every lab assignment, all logs must be included in the submission. This log and videos will be used to fairly give grades to all members of the group.

Please upload a single edited video of your work to YouTube. Then generate and submit a PDF that includes a link to your YouTube video and the GitHub logs.

In this lab, you will explore the real-world implications of how TCP performs under less-than-perfect conditions. Your goal is to develop a file transfer program (like scp or ftp) that uses your custom protocol and performs better than any other program available. This is your first competitive laboratory. The top 25% of the teams with the highest performance will receive Extra credit. The program should be built for the Linux OS, and code your program using C or C++.

While TCP is effective, reliable, and relatively robust on the Internet, it doesn't always give us the best throughput under every circumstance. In this section, you'll design an IP-based file-transfer utility. The design and implementation of the utility is up to your group; however, it must fulfill only three requirements: it must use IP (so it can be routed), it must transfer the file reliably (with no errors), and it must be implemented with a command-line interface similar to scp.

The link speed of the sender and receiver must be 100Mbps, and the test file size must be at least 1 GB. You should emulate the delay and the loss rate of the link using the delay node. You should test your system under various conditions. However, the three mandatory conditions for the assignment are:

- Case 1: The round-trip time (RTT) of 10ms with the Loss rate of 1% (bi-directional) on a network configured to transfer at 100Mbps/sec for the server, client, and router.
- Case 2: The RTT of 200ms with the Loss rate of 20% (bi-directional) on a network configured to transfer at 100Mbps/sec for the server, client, and router.
- Case 3: The RTT of 200ms without any packet loss (bi-directional) on a network configured to transfer at 100Mbps/sec for the server and client. On the router, the network throughput is limited to 80Mbps/sec.

You should run the above tests with an MTU of 1500 and then with an MTU of 9000.

For this lab, use two nodes connected through VyOS router that can be configured to add corresponding **delays of RTT and the loss rates**. These settings should be verified using tools like iperf and ping. The results of the system may vary depending on your computer nodes' performance. **This lab can be completed using virtual machines or 3 cloud computing nodes connected by the network.** The goal of the lab is to obtain the best file transfer performance.

1. Simulating Networking Environments

Set a rate limit on both server and client to restrict speed to 100Mbit/s on egress by executing the below command on both server and client. Here, it's assumed that eth0 belongs to the 10.200.1.0/24 subnet for the client and eth0 belongs to the 10.200.2.0/24 subnet for the server, respectively.

Egress rate limiting:

```
# sudo tc qdisc add dev eth0 root tbf rate 100mbit latency 0.001ms burst 9015
```

Now set the rate limit to 100Mbit/sec on the egress path of the router on two network interfaces by executing the command below. Here, it's assumed that eth0 belongs to 10.200.1.0/24 subnet and eth1 belongs to 10.200.2.0/24 subnet.

Egress rate limiting

```
# sudo tc qdisc add dev eth0 root handle 1:0 tbf rate 100mbit latency 0.001ms burst 9015
# sudo tc qdisc add dev eth1 root handle 1:0 tbf rate 100mbit latency 0.001ms burst 9015
```

First, ensure that rate limiting is working by executing iperf. You need to get a TCP throughput of around 100 Mbits/sec. Also, check the bandwidth on UDP mode to be around 100Mbits/sec (iperf -u -c <dst_ip> -b 200mbit).

```
ubuntu@ip-10-200-2-48: ~
ubuntu@ip-10-200-2-48:~$ iperf -c 10.200.1.83
-----
Client connecting to 10.200.1.83, TCP port 5001
TCP window size: 85.0 KByte (default)
-----
[  3] local 10.200.2.48 port 33652 connected with 10.200.1.83 port 5001
[ ID] Interval           Transfer     Bandwidth
[  3]  0.0-10.0 sec      112 MBytes   93.5 Mbits/sec
ubuntu@ip-10-200-2-48:~$
```

```
ubuntu@ip-10-200-2-48:~$ iperf -u -c 10.200.1.83 -b 200mbit
-----
Client connecting to 10.200.1.83, UDP port 5001
Sending 1470 byte datagrams
UDP buffer size:  208 KByte (default)
-----
[  3] local 10.200.2.48 port 44832 connected with 10.200.1.83 port 5001
[ ID] Interval           Transfer     Bandwidth
[  3]  0.0-10.0 sec       242 MBytes   203 Mbits/sec
[  3] Sent 172412 datagrams
[  3] Server Report:
[  3]  0.0-10.3 sec      115 MBytes   94.4 Mbits/sec  15.668 ms 90103/172411 (52%)
[  3]  0.0-10.3 sec      1 datagrams received out-of-order
ubuntu@ip-10-200-2-48:~$
```

Now, set the delay of RTT 10ms and the loss rate of 1% in both tx and rx by executing the below command on both eth0 and eth1 network interfaces of the VyOS (router). Here eth0 has 10.200.1.0/24 and eth1 has

10.200.2.0/24 or vice versa. The following commands simulate a packet drop with high propagation delay on the link.

Case 1: RTT of 10ms, Packet loss of 1%

```
# sudo tc qdisc add dev eth0 parent 1:1 handle 10: netem delay 5ms drop 1%
# sudo tc qdisc add dev eth1 parent 1:1 handle 10: netem delay 5ms drop 1%
```

Case 2: RTT of 200ms, Packet loss of 20%

```
# sudo tc qdisc change dev eth0 parent 1:1 handle 10: netem delay 100ms drop 20%
# sudo tc qdisc change dev eth1 parent 1:1 handle 10: netem delay 100ms drop 20%
```

From ping command: (execute on server to ping to client and also from client to ping to server)

```
# ping -i 0.2 -c 200 <dest_ip>
```

Ensure that you are seeing the delay and drop rate which you have configured at the router in the ping output, with the drop rate within the range of $\pm 20\%$ from the set value, and the delay value slightly greater than the set value. A snapshot is given below as a reference. (This is for case 2)

```
64 bytes from 10.200.1.83: icmp_seq=194 ttl=63 time=201 ms
64 bytes from 10.200.1.83: icmp_seq=196 ttl=63 time=201 ms
64 bytes from 10.200.1.83: icmp_seq=197 ttl=63 time=200 ms
64 bytes from 10.200.1.83: icmp_seq=199 ttl=63 time=201 ms

--- 10.200.1.83 ping statistics ---
200 packets transmitted, 126 received, 37% packet loss, time 40450ms
rtt min/avg/max/mdev = 200.906/201.086/202.648/0.319 ms, pipe 2
ubuntu@ip-10-200-2-48:~$
```

You would also need to confirm the packet drop rate on UDP mode in both directions with iperf. (start server iperf on client and run it, and then start it on server and run it) (iperf -u -c <dst_ip> -b 100mbit) with $\pm 20\%$ variation allowed on drop rate. (This is for case 2). You should see an approximate drop of 20% with $\pm 20\%$ variation on the value.

Server => Client:

```
ubuntu@ip-10-200-2-48:~$ iperf -u -c 10.200.1.83 -b 100mbit
-----
Client connecting to 10.200.1.83, UDP port 5001
Sending 1470 byte datagrams
UDP buffer size: 208 KByte (default)
-----
[  3] local 10.200.2.48 port 41704 connected with 10.200.1.83 port 5001
[ ID] Interval           Transfer     Bandwidth
[  3]  0.0-10.0 sec      120 MBytes  101 Mbits/sec
[  3] Sent 85471 datagrams
[  3] Server Report:
[  3]  0.0-10.3 sec     92.8 MBytes  75.9 Mbits/sec  15.723 ms 19307/85470 (23%)
[  3]  0.0-10.3 sec      1 datagrams received out-of-order
ubuntu@ip-10-200-2-48:~$
```

Client => server:

```

ubuntu@ip-10-200-1-83:~$ iperf -u -c 10.200.2.48 -b 100mbit
-----
Client connecting to 10.200.2.48, UDP port 5001
Sending 1470 byte datagrams
UDP buffer size: 208 KByte (default)
-----
[  3] local 10.200.1.83 port 48437 connected with 10.200.2.48 port 5001
[ ID] Interval           Transfer         Bandwidth
[  3]  0.0-10.0 sec      120 MBytes      101 Mbits/sec
[  3] Sent 85470 datagrams
[  3] Server Report:
[  3]  0.0-10.0 sec      92.6 MBytes      77.7 Mbits/sec    0.098 ms 19414/85469 (23%)
[  3]  0.0-10.0 sec      1 datagrams received out-of-order
ubuntu@ip-10-200-1-83:~$

```

TCP iperf:

```

ubuntu@ip-10-200-2-48:~$ iperf -c 10.200.1.83
-----
Client connecting to 10.200.1.83, TCP port 5001
TCP window size: 85.0 KByte (default)
-----
[  3] local 10.200.2.48 port 33690 connected with 10.200.1.83 port 5001
[ ID] Interval           Transfer         Bandwidth
[  3]  0.0-23.9 sec      384 KBytes       132 Kbits/sec
ubuntu@ip-10-200-2-48:~$

```

You will also need to create a 1GB file on your sender computer. You can create this from the steps mentioned in Lab3 of scp part.

```
# dd if=/dev/urandom of=data.bin bs=1M count=1024
```

Case-3: RTT of 200ms, No packet loss, but the network transfer rate is reduce by 20%

Delete previous set configurations on qdisc of all nodes (verify it using `sudo tc qdisc show`). Now set the rate limit to 100Mbps/sec on the server and client VM. On Router VM, set the rate limit to 80Mbps/sec with RTT of 200ms. There is no drop configured here in this step. Ensure that ping shows a delay of 200ms and the rate is limited to around 80Mbps/sec for the UDP test in iperf. with no packet drop visible in ping.

Router command:

Egress rate-limiting:

```
# sudo tc qdisc add dev eth0 root handle 1:0 tbf rate 80mbit latency 0.001ms burst 9015
# sudo tc qdisc add dev eth1 root handle 1:0 tbf rate 80mbit latency 0.001ms burst 9015
```

Egress delay add in tx, rx:

```
# sudo tc qdisc add dev eth0 parent 1:1 handle 10: netem delay 100ms
# sudo tc qdisc add dev eth1 parent 1:1 handle 10: netem delay 100ms
```

Server, client egress rate limit command:

```
# sudo tc qdisc add dev eth0 root tbf rate 100mbit latency 0.001ms burst 9015
```

```

ubuntu@ip-10-200-2-48:~$ iperf -u -c 10.200.1.83 -b 100mbit
-----
Client connecting to 10.200.1.83, UDP port 5001
Sending 1470 byte datagrams
UDP buffer size: 208 KByte (default)
-----
[  3] local 10.200.2.48 port 46053 connected with 10.200.1.83 port 5001
[ ID] Interval      Transfer    Bandwidth
[  3] 0.0-10.0 sec  120 MBytes  101 Mbits/sec
[  3] Sent 85471 datagrams
[  3] Server Report:
[  3] 0.0-10.1 sec  93.2 MBytes  77.8 Mbits/sec   0.111 ms 18984/85469 (22%)
[  3] 0.0-10.1 sec  1 datagrams received out-of-order
ubuntu@ip-10-200-2-48:~$

```

```

ubuntu@ip-10-200-2-48: ~
ubuntu@ip-10-200-2-48:~$ iperf -c 10.200.1.83
-----
Client connecting to 10.200.1.83, TCP port 5001
TCP window size: 85.0 KByte (default)
-----
[  3] local 10.200.2.48 port 33688 connected with 10.200.1.83 port 5001
[ ID] Interval      Transfer    Bandwidth
[  3] 0.0-10.9 sec  3.38 MBytes  2.60 Mbits/sec
ubuntu@ip-10-200-2-48:~$

```

Critical Thinking

As you can observe in Case 2 and Case 3, throughputs of iperf UDP are similar (~78Mbps) but TCP differ by up to 20x. Think about your configuration and describe in the report exactly what might be happening for Case 2 and Case 3 in terms of network buffering, packet loss, latencies, and window sizes.

2. Custom Fast and Reliable Protocol Design

By now, you should have observed interesting UDP/TCP network protocol performance results under different environments. The most interesting result is the way the TCP protocol fails to transfer anywhere close to 80Mbps when the network has high latency with 20% packet loss.

The goal of this step of the lab is to write a file transfer program that will reliably transfer the 1-gigabyte file from the sender/client to the receiver/server at the highest performance possible through all three **network environments described above**. The file transfer application should be written using the socket library that you used in the previous lab.

Also, ensure that your file transfer application can work with different MTUs of the interface. **Test it on MTU of 1500 and then on MTU of 9001**. MTU needs to be set to the same value for all the network interfaces belonging to the server, client, and VyOS. Your client file transfer application should save the file in the location specified on the command line. Similarly, the server part of the application should send the file specified on the command line.

You must timestamp right when the first bit of the data leaves the sender, and the final bit is received by the receiver. The total time for the file to make the one-way trip will be used to compare your program's performance against other teams' program performance. You will also need to prove the reliability of your system by **running MD5 on both the original file and the received file**. You will need to run your ftp application against MTUs of 1500 and 9001 on two specified drop rates, delay, and rate limits as mentioned

above for three cases. (Repeat the same procedure, one with MTU of 1500 and then with 9001, and observe the file transfer time)

The execution of the system should be straightforward, and any notion of cheating will result in automatic zero. The goal of this task is to encourage a healthy and competitive development environment for everyone. You are encouraged to help each other to get the best result

- 1) The minimum transfer rate for your final application is 20 Mbps to compete against the other team.
- 2) Describe, in detail, the concept(s) behind your file transfer utility, results, flow chart, data structure, and algorithm used, and the analysis in the report document that must be submitted along with source files on the due date and time.
- 3) The scoring will be based on the submitted documents

REFERENCE

<https://computing.llnl.gov/tutorials/pthreads/> (multi-threading)
<https://www.geeksforgeeks.org/data-structures/> (data- structure)
<https://www.geeksforgeeks.org/fundamentals-of-algorithms/> (algorithms)